

Building a Large-Scale Instagram Public Data Intelligence App

Executive summary

A product that looks like **Dolphin Radar for agencies** breaks into two very different businesses. The first is a relatively durable, lower-risk platform for **connected Instagram professional accounts** and **public professional-account intelligence**: profile metadata, media, account and media insights, comments and mentions on owned media, messaging events, hashtag discovery, creator/brand research, and limited ad intelligence through Meta's official surfaces. Meta's current Instagram APIs are explicitly centered on **professional accounts** (businesses and creators), and the legacy Instagram Basic Display API was shut down on **December 4, 2024**. Meta's current documentation also shows Business Discovery for **other professional accounts**, Public Content Access for **hashtag search**, webhooks for **comments/mentions/messages**, and April 2026 updates that added the ability to **like/unlike media and comments on behalf of an app user**. ¹

The second business is a high-risk "public user activity tracker" product for **arbitrary target accounts**, especially features like **last follow/unfollow**, **recent likes across other accounts**, **who they followed most recently**, and **outbound interactions from a target user to others**. Those capabilities are heavily marketed by tools such as Dolphin Radar and Snoopreport, but they are **not features Meta publicly documents in its official Instagram APIs for arbitrary third-party public users**. Meta's own Instagram Terms say users may not access or collect information in unauthorized ways, and Instagram explicitly states that data scraping violates its Terms. In practical terms, that means a full Dolphin-Radar-style replica for arbitrary public accounts almost certainly depends on **scraping and/or third-party scraped data**, even if the data is public. ²

That distinction should drive product strategy. The strongest recommendation is to build a **two-lane architecture**: a compliant production lane for official and consented data, and a strictly segregated, policy-gated "public intelligence" lane that is limited to what your legal counsel and distribution channels will tolerate. For a mainstream SaaS sold to agencies, the best long-run product is **official-first**: connected-account analytics, competitor intelligence on public professional accounts, creator discovery, campaign tracking, audience/engagement benchmarking, comment and mention workflows, and ad visibility via official Meta surfaces where available. ³

From a systems perspective, the best large-scale design is a **hybrid OLTP/OLAP architecture**: PostgreSQL for core entities, tenancy, permissions, jobs, and transactional state; ClickHouse for append-heavy event analytics and real-time dashboard queries; S3-compatible object storage for raw payloads, exports, and backfills; Redis for caching and distributed rate limiting; and Kafka-compatible streaming for ingestion and downstream fan-out. That recommendation aligns with the documented strengths of PostgreSQL logical replication and partitioning, ClickHouse's columnar OLAP design and materialized views, Redis-based rate limiting, and Kafka-compatible event streaming platforms such as Kafka or Redpanda. ⁴

Commercially, the biggest cost driver is usually **data acquisition**, not compute. Public vendor list prices already show why: Bright Data advertises Instagram datasets at **\$250 per 100,000 records**, while Apify's maintained Instagram scrapers are priced from about **\$1.00–\$1.50 per 1,000 results/posts**. Infrastructure can often start in the low thousands of dollars per month; continuous public-account surveillance at scale can push data acquisition alone into the **tens of thousands per month**. ⁵

Market reality and feature inventory

The current market splits into a small set of “activity tracker” products that claim to expose public follow/like/comment behaviors, and a larger set of creator analytics, influencer vetting, social listening, and follower-growth products. Dolphin Radar positions itself as an all-in-one Instagram tracker for **public interactions, likes, followers, recent follow activity**, and in one feature page it says it tracks **new follows, unfollows, post likes, comments, story engagement, and follower count changes**. Snoopreport similarly claims weekly or monthly reports covering **liked photos/Reels, followed/unfollowed profiles**, and activity patterns. By contrast, Modash, HypeAuditor, and Social Blade are much more oriented toward **creator discovery, engagement, audience analytics, and growth tracking** than hidden activity reconstruction. ⁶

Competitive feature inventory

Product	Recent follows / unfollows	Likes / comment activity	Engagement / growth analytics	Campaign / creator workflow	Ads / paid content visibility	Notes	Source
Dolphin Radar	Yes	Yes	Yes	Partial	Not clearly documented	Claims tracking of public follows, unfollows, likes, comments, follower changes, history, and AI-generated analysis.	⁷
Snoopreport	Yes	Yes	Yes	Partial	Not clearly documented	Weekly/monthly reports; liked posts, followed/unfollowed accounts, interests, activity timeline, hashtags.	⁸

Product	Recent follows / unfollows	Likes / comment activity	Engagement / growth analytics	Campaign / creator workflow	Ads / paid content visibility	Notes	Source
Inflact	Not prominently documented	Partial	Yes	Partial	Not clearly documented	Focuses on profile analyzer, follower analytics, posting cadence, top posts, engagement.	9
Modash	No public claim like Dolphin Radar	No public claim like Dolphin Radar	Yes	Yes	Partial	End-to-end influencer platform: discovery, vetting, content/ performance tracking, campaign metrics.	10
HypeAuditor	No public claim like Dolphin Radar	Partial	Yes	Yes	Partial	Strong on audience demographics, fraud detection, engagement quality, growth tracking.	11
Social Blade	No	No	Yes	No	No	Daily analytics, progress charts, projections, live follower counts.	12

The practical takeaway is that there is real market demand for “what did this public account just do?” monitoring, but the more durable market segment is **analytics and operations**, not covert behavior tracking. Agencies are more likely to pay for **campaign measurement, creator vetting, competitor benchmarking, comments and mentions triage, ad intelligence, branded-content discovery, and account growth signals** than for person-level follow surveillance. That is also the segment best aligned with official Meta products such as the Instagram APIs, Creator Marketplace API, messaging/comments APIs, and Business Discovery on professional accounts. ¹³

Feasibility by requested feature

Requested feature	Officially feasible today	Scope	Commentary	Source
Recent posts from a connected professional account	Yes	Owned business/ creator account	Supported through Instagram APIs for professional accounts and media endpoints.	14
Recent posts from another public professional account	Partial	Other business/ creator accounts	Business Discovery supports basic metadata and metrics about other professional accounts.	15
Comments on owned media	Yes	Owned media	Meta documents comments APIs and real-time Instagram webhooks for comments.	16
Mentions inbound to owned account	Yes	Owned professional account	Webhooks and mentions endpoints exist; story mentions have limitations.	17
DMs inbound to owned account	Yes	Owned professional account	Messaging API and webhooks are documented.	18
Outbound comments / likes by target user to others	Partial	Owned professional account only	Comment management exists; like/unlike on behalf of app users was added in 2026.	19
Arbitrary user's last follow / unfollow	No public official endpoint documented	Arbitrary public user	This is the signature "tracker" feature, but not something Meta documents for arbitrary third-party accounts.	20
Arbitrary user's like history across Instagram	No public official endpoint documented	Arbitrary public user	Meta added write-side like/unlike for app users, not a general read API for arbitrary public users' like histories.	21
Personal-account data via official Instagram consumer API	No	Personal accounts	Basic Display API was deprecated and shut down; Meta's current APIs focus on professional accounts.	22

Requested feature	Officially feasible today	Scope	Commentary	Source
Ads intelligence	Partial	Owned ads and some public transparency data	Marketing API covers owned ad workflows; Ad Library API has limited public archive scope.	23

Data sources and collection strategy

Meta's official surface area is useful, but narrower than many founders initially expect. The Instagram Platform overview says the APIs are for **Instagram professional accounts**, including both businesses and creators. Business Discovery lets an app get **basic metadata and metrics about other Instagram professional accounts**, including follower and media counts. Public Content Access is scoped to **Hashtag Search endpoints**, requires App Review, and may require business verification and additional contracts. Real-time webhooks exist for comments, mentions, and messaging; they do **not** create a general-purpose "follow/unfollow firehose" for arbitrary users. 24

The "ads" side is similarly constrained. Meta's Ad Library API officially supports customized searches for **social issues/elections/politics ads delivered anywhere in the world during the past seven years**, and **ads of any type delivered to the UK or EU during the past year**. Marketing API and Instagram ads surfaces are much richer for **owned advertising accounts**, including comment handling on ad media through the ad's `effective_instagram_media_id`. That means global competitor ad intelligence is only partly available through official public APIs; outside the documented archive scope, many tools rely on human review or public-page capture of Ad Library pages rather than a clean global API. 25

Collection methods compared

Method	Coverage	Latency	Reliability	Policy risk	Best use	Source
Instagram APIs with Instagram Login / Facebook Login	Owned pro accounts; some discovery of other pro accounts	Near real time for webhooks; API pull otherwise	Highest	Lowest	Core agency SaaS; connected-account analytics and workflows	26
Hashtag Search / Public Content Access	Hashtag-centered discovery only	Near real time to hourly	High once approved	Low to moderate	Brand monitoring, campaign hashtags, UGC discovery	27

Method	Coverage	Latency	Reliability	Policy risk	Best use	Source
Marketing API / owned ad assets	Owned ad accounts and related media/ comment flows	Near real time	High	Low	Paid social reporting and moderation for customers' own ads	28
Meta Ad Library API / Ads Archive	Public archive with documented regional/ category limits	Hours to daily	High for its stated scope	Low	Public ad intelligence where the official archive covers it	29
Third-party user-permissioned providers	Usually professional/ creator or consented account data	API dependent	Medium to high	Moderate	Faster integrations where users explicitly connect accounts	30
Third-party scraped datasets / scraper APIs	Broad public pages, comments, profiles, hashtags, sometimes likes	Near real time to batch	Variable	High	Non-core enrichment, controlled experiments, backfills	31
In-house scraping of public pages	Broadest potential public coverage	Depends on implementation	Lowest over time	Highest	Only for legally reviewed, narrow, explicitly accepted risk areas	32

Recommended acquisition strategy

The best production strategy is **official-first, provider-second, scraping-last**.

For the first year, the recommended product surface is:

1. **Connected professional account analytics** via Instagram APIs with Instagram Login and/or Facebook Login.

2. **Competitor / creator intelligence on public professional accounts** via Business Discovery.
3. **Hashtag and UGC discovery** via Public Content Access.
4. **Owned ads and public ad archive intelligence** via Marketing API and Ad Library/Ads Archive where applicable.
5. **Optional vendor enrichment** for public professional profiles/posts if a customer contract and legal review permit it.

That mix covers most agency workflows while avoiding the most brittle and controversial activity-tracking features. The major exception is if the business thesis truly depends on “someone’s last follow/unfollow.” In that case, the report’s recommendation is to run that as a **separate risk-isolated product line**, with its own terms, access controls, vendor management, logging, takedown response, and jurisdictional rollout rules.

33

Legal, privacy, and platform constraints

The hardest problem is not technical; it is governance. Instagram’s Terms say users cannot access or collect information in unauthorized ways, and Instagram states directly that data scraping goes against its Terms of Use. Meta’s Platform Terms and recent developer notices also emphasize app review, advanced access, data deletion obligations, and compliance monitoring. In November 2024 Meta reiterated that, under its Platform Terms, developers must **promptly delete user data on request**. 34

Public availability does **not** eliminate privacy law exposure. EU/UK GDPR principles emphasize **purpose limitation, data minimisation, and storage limitation**; California’s CCPA/CPRA provides **deletion rights** and rights around disclosure and sharing. If you store account-level interaction histories that can be associated with identifiable people, those principles become operational requirements: collect only what is necessary, document lawful basis and purpose, define retention windows, support deletion workflows, and restrict internal access. 35

The current U.S. legal picture is mixed rather than permissive. Courts have narrowed some anti-hacking and contract theories against logged-off scraping of public data, including the Ninth Circuit’s 2022 hiQ ruling discussed by Reuters and the 2024 Meta v. Bright Data decision relied on by legal commentary. But those decisions do **not** create a blanket right to run a commercial surveillance product. Contract risk, privacy claims, platform retaliation, app-store review risk, anti-bot claims, unfair competition claims, and non-U.S. data protection obligations can still be material. Reuters also reported in April 2026 that an Italian court allowed a class action against Meta relating to a prior data scraping incident, which underlines how privacy exposure can survive even if some U.S. access theories are narrowed. 36

The safest interpretation is this: **what may be reachable on the public web is not automatically low-risk to aggregate, analyze, monetize, and retain at scale**. That is even more true for products marketed to individuals to watch other individuals. Snoopreport’s own marketing examples include partners, children, celebrities, and competitors; that use-profile may be sellable, but it is exactly the kind of profile that raises elevated privacy, trust-and-safety, and distribution risk. 37

Risk mitigation strategy

A robust mitigation program should include the following design rules:

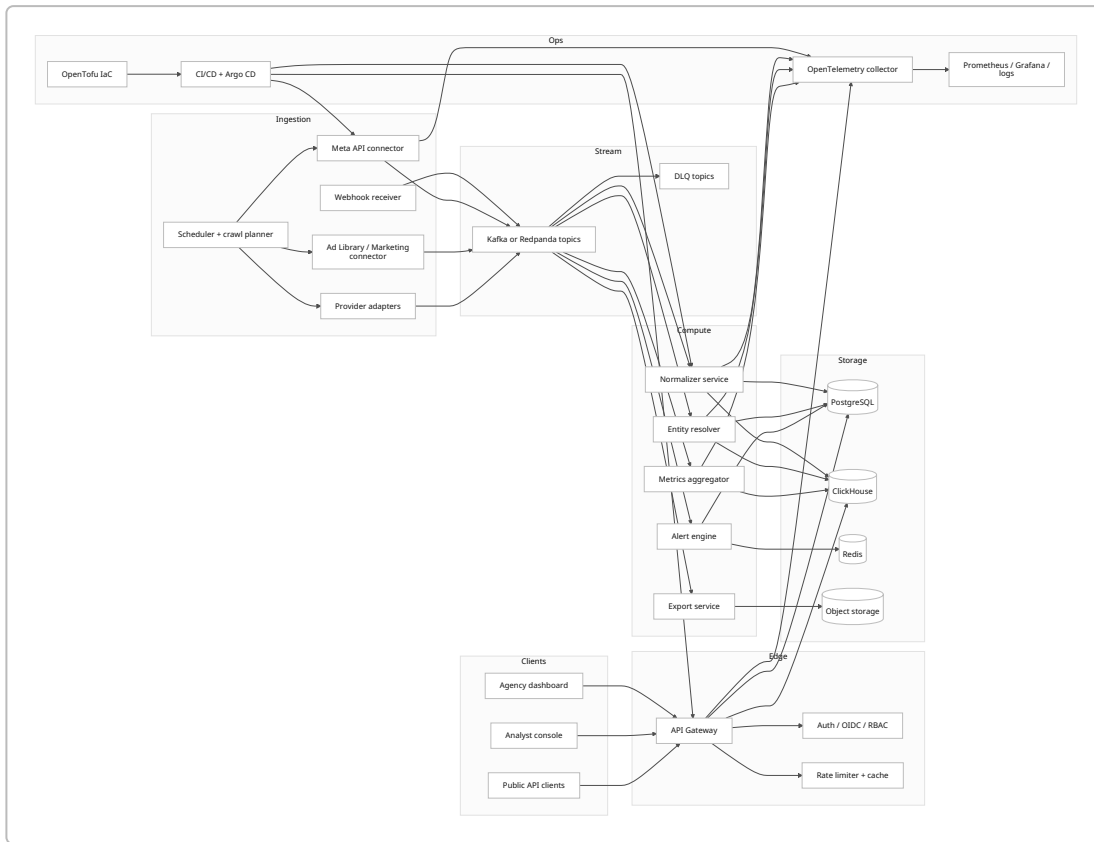
Control area	Recommended policy
Product scope	Make the default product about brands, creators, businesses, and connected client accounts , not covert monitoring of ordinary individuals.
Source segregation	Keep official/consented data in a separate pipeline, schema, and access layer from scraped/public-intelligence data.
Jurisdiction gating	Disable or narrow high-risk features in stricter jurisdictions unless counsel signs off.
Data minimization	Do not store raw HTML/screenshots longer than necessary; normalize to the smallest useful event schema.
Deletion and takedowns	Support verifiable deletion requests, account suppression lists, source blocks, and provider-level takedown propagation.
Retention	Use short retention on raw payloads, medium retention on identifiable event logs, long retention only on de-identified aggregates.
Abuse prevention	Detect repeated lookup of private individuals, stalking-like query patterns, and employee misuse via immutable audit logs.
Customer terms	Prohibit harassment, surveillance, intimate-partner abuse, and unlawful profiling in customer contracts and onboarding.

Those controls are not optional extras; they are what turn a technically impressive ingestion system into a defensible business. They are directly motivated by Meta's scraping restrictions, data deletion obligations, and core privacy-law principles. ³⁸

Reference architecture and recommended stack

The reference system should treat Instagram data as an **event ingestion and analytics problem**, not as a classic CRUD app. The fastest path is a Rust-heavy service tier, PostgreSQL as the source of truth for entities and workflows, ClickHouse for analytics, Redis for latency-critical caching and quota enforcement, S3 for raw storage, and Kafka-compatible streaming for ingestion fan-out. ClickHouse is explicitly built for real-time analytical reporting in SQL on high-ingest columnar datasets; PostgreSQL provides mature relational guarantees plus logical replication; BRIN indexes and partitioning are well-suited to very large append-correlated tables; Redis is a natural fit for distributed rate limiting; and Kafka-compatible brokers remain the safest choice for durable event backbones. ³⁹

High-level architecture diagram



Database options compared

Option	Strengths	Weaknesses	Recommended role
PostgreSQL	Excellent OLTP, mature indexing, JSONB, logical replication, partitioning	Becomes costly for very large analytical scans if used alone	Source of truth for tenants, accounts, jobs, permissions, alerts, APIs
TimescaleDB	PostgreSQL-compatible time-series/event optimization via hypertables	Still row-store economics; less ideal than ClickHouse for huge multidimensional analytics	Good substitute for a lower-complexity build under moderate scale
ClickHouse	Columnar real-time OLAP, high-ingest analytics, materialized views, efficient partition pruning	Separate operational surface; not your primary transactional store	Main analytics engine for timelines, rollups, cohorts, top-Ns, dashboard cards
DuckDB	Fast embedded OLAP, excellent for notebooks and one-off backfills	Not a multi-tenant serving database	Analyst workbench, QA, reproducible offline investigations

This combination is preferable because it lets each engine do what it is designed for: PostgreSQL for relational correctness and workflows, ClickHouse for event-heavy analytics, and DuckDB for local/offline analyst workflows. ⁴⁰

Message broker options compared

Broker	Strengths	Weaknesses	Recommendation
Apache Kafka	Mature ecosystem, exactly-once support in streams, best fit for CDC and analytics pipelines	Highest operational weight	Best default if you expect CDC, heavy connectors, and large replay needs
Redpanda	Kafka-client compatibility, simpler operational model, no ZooKeeper, lower ops burden	Smaller ecosystem than Kafka proper	Strong default for most startups that want Kafka semantics with less ops
NATS JetStream	Very lightweight, low-latency, good control-plane/event fan-out	Less natural than Kafka for large replay-heavy analytics pipelines	Use for control-plane or lightweight eventing, not as the primary analytics spine

The recommended default is **Redpanda** for a startup team and **Kafka** if you already know you need the deepest connector/CDC ecosystem. ⁴¹

Scraping and provider approaches compared

Approach	Advantages	Drawbacks	Use only if
Official APIs only	Best durability, simplest compliance story, easiest app review story	Cannot replicate Dolphin-Radar-style arbitrary public-user activity tracking	You target agencies, creators, brands, and connected accounts
Provider-assisted public data	Faster time to market, less in-house scraper maintenance	Vendor lock-in, data provenance concerns, cost grows fast	You need narrow enrichment beyond official APIs
In-house public-page scraping	Maximum control, potentially maximum feature reach	Highest legal/policy risk and maintenance burden	It is a ring-fenced, legally reviewed module with explicit business justification

Recommended tech stack and libraries

The stack below is opinionated for a large Rust-centric application:

Layer	Recommendation	Why
Frontend	Next.js + TypeScript + TanStack Query + Tailwind + shadcn/ui + ECharts	Mature data-heavy dashboard stack, fast SSR/app-router UX, strong table/chart ecosystem
API gateway	Rust + axum + tower + tokio	<code>axum</code> is ergonomic and modular; Tokio is the standard Rust async runtime for production services.
Domain services	Rust + axum or tonic gRPC	Performance, type safety, good fit for IO-heavy ingestion and worker services
DB access	sqlx	Compile-time checked SQL against real schemas is excellent for a data product with many migrations.
Streaming	Redpanda/Kafka + rdkafka	Durable replay, partitioned consumer groups, CDC compatibility
Cache and quotas	Redis	Excellent distributed cache and rate-limiter substrate
Analytics DB	ClickHouse	Real-time OLAP and rollups
OLTP DB	PostgreSQL	Transactional correctness and tenancy
CDC / ETL	Debezium	Proven CDC model from database change logs into topics
Observability	OpenTelemetry + Prometheus + Grafana + centralized logs	Unified traces, metrics, and logs with vendor neutrality
IaC / CD	OpenTofu + Argo CD + GitHub Actions	Strong GitOps workflow with versioned infra and auditable deploys

The Rust choices are especially well aligned with long-lived ingestion workers, concurrent API clients, and low-overhead API services. `axum` focuses on modular HTTP request handling, Tokio provides async scheduling and I/O primitives, and `sqlx` supports compile-time checked queries. ⁴²

Security, compliance, observability, and deployment

For security, use OIDC for human users, short-lived service credentials for machines, envelope encryption with KMS/HSM-backed keys, row ownership checks at the service layer plus optional PostgreSQL RLS, signed webhooks, immutable audit trails, and tenant-scoped export signing. For compliance, plan early for privacy intake workflows, deletion/takedown queues, customer DPA support, and a documented data inventory. For observability, instrument every ingress, normalization, and query path with OpenTelemetry traces and metrics, then publish service-level objectives around ingest freshness, webhook success, API latency, and dashboard freshness. For delivery, GitHub Actions plus OpenTofu plus Argo CD gives a clean pipeline from commit to environment, with Git as the source of truth for both application and cluster state. ⁴³

Data model, APIs, and operator experience

The core modeling choice is to separate **entities**, **snapshots**, and **events**. Entities are durable records such as accounts, media, tenants, rules, providers, and jobs. Snapshots capture state at a point in time, such as follower count or engagement metrics. Events capture discrete facts such as a new comment, an inbound mention, a detected likely follow edge change, or a provider refresh completion. PostgreSQL partitioning and BRIN indexes work well for large append-correlated tables, while ClickHouse MergeTree tables and materialized views are excellent for serving rollups and real-time dashboards. 44

Canonical PostgreSQL schema

The following SQL is a pragmatic foundation. In production, the large append tables should be partitioned monthly.

```
CREATE TABLE tenants (  
  id          BIGSERIAL PRIMARY KEY,  
  slug        TEXT NOT NULL UNIQUE,  
  name        TEXT NOT NULL,  
  plan_code   TEXT NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE users (  
  id          BIGSERIAL PRIMARY KEY,  
  email       CITEXT NOT NULL UNIQUE,  
  full_name   TEXT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE tenant_memberships (  
  tenant_id   BIGINT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  user_id     BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  role_code   TEXT NOT NULL CHECK (role_code IN  
( 'owner', 'admin', 'analyst', 'viewer' )),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (tenant_id, user_id)  
);  
  
CREATE TABLE instagram_accounts (  
  id          BIGSERIAL PRIMARY KEY,  
  tenant_id   BIGINT NOT NULL REFERENCES tenants(id) ON DELETE  
CASCADE,  
  platform_account_id TEXT,  
  username    TEXT NOT NULL,  
  account_type TEXT NOT NULL CHECK (account_type IN  
( 'business', 'creator', 'personal', 'unknown' )),
```

```

        is_connected          BOOLEAN NOT NULL DEFAULT false,
        is_public              BOOLEAN NOT NULL DEFAULT true,
        biography              TEXT,
        website_url            TEXT,
        profile_picture_url    TEXT,
        followers_count_latest BIGINT,
        follows_count_latest   BIGINT,
        media_count_latest     BIGINT,
        source_of_truth         TEXT NOT NULL CHECK (source_of_truth IN
('meta_api','provider','scrape','mixed')),
        first_seen_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
        last_seen_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
        UNIQUE (tenant_id, username)
);

CREATE INDEX idx_instagram_accounts_tenant_username
ON instagram_accounts (tenant_id, username);

CREATE TABLE account_snapshots (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
CASCADE,
    instagram_account_id BIGINT NOT NULL REFERENCES instagram_accounts(id) ON
DELETE CASCADE,
    captured_at       TIMESTAMPTZ NOT NULL,
    followers_count    BIGINT,
    follows_count      BIGINT,
    media_count        BIGINT,
    engagement_rate    NUMERIC(8,4),
    avg_likes_12       NUMERIC(18,2),
    avg_comments_12    NUMERIC(18,2),
    source_name        TEXT NOT NULL,
    source_run_id      TEXT
);

CREATE INDEX idx_account_snapshots_account_time
ON account_snapshots (instagram_account_id, captured_at DESC);

CREATE TABLE media_items (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
CASCADE,
    instagram_account_id BIGINT NOT NULL REFERENCES instagram_accounts(id) ON
DELETE CASCADE,
    platform_media_id TEXT,
    media_type        TEXT NOT NULL CHECK (media_type IN
('image','video','carousel','reel','story','unknown')),
    shortcode         TEXT,

```

```

    permalink            TEXT,
    caption              TEXT,
    published_at         TIMESTAMPTZ,
    like_count_latest    BIGINT,
    comment_count_latest BIGINT,
    view_count_latest    BIGINT,
    is_sponsored         BOOLEAN,
    raw_json             JSONB,
    first_seen_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, platform_media_id)
);

CREATE INDEX idx_media_items_account_published
    ON media_items (instagram_account_id, published_at DESC);

CREATE TABLE media_metric_snapshots (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
CASCADE,
    media_item_id     BIGINT NOT NULL REFERENCES media_items(id) ON DELETE
CASCADE,
    captured_at       TIMESTAMPTZ NOT NULL,
    like_count        BIGINT,
    comment_count     BIGINT,
    view_count        BIGINT,
    saved_count       BIGINT,
    reach_count       BIGINT,
    impressions_count BIGINT,
    source_name       TEXT NOT NULL
);

CREATE INDEX idx_media_metric_snapshots_media_time
    ON media_metric_snapshots (media_item_id, captured_at DESC);

CREATE TABLE interaction_events (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
CASCADE,
    subject_account_id BIGINT NOT NULL REFERENCES instagram_accounts(id)
ON DELETE CASCADE,
    related_account_username TEXT,
    media_item_id     BIGINT REFERENCES media_items(id) ON DELETE SET
NULL,
    event_type        TEXT NOT NULL CHECK (
        event_type IN (
            'comment_inbound',
            'mention_inbound',

```

```

        'message_inbound',
        'comment_outbound',
        'like_outbound',
        'follow_edge_added',
        'follow_edge_removed',
        'ad_seen',
        'snapshot_refresh'
    )
),
event_time          TIMESTAMPTZ NOT NULL,
confidence_score     NUMERIC(5,4) NOT NULL DEFAULT 1.0,
source_name          TEXT NOT NULL,
payload              JSONB
);

CREATE INDEX idx_interaction_events_subject_time
ON interaction_events (subject_account_id, event_time DESC);

CREATE INDEX idx_interaction_events_tenant_type_time
ON interaction_events (tenant_id, event_type, event_time DESC);

CREATE TABLE collection_jobs (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
CASCADE,
    target_type        TEXT NOT NULL CHECK (target_type IN
('account','media','hashtag','ad_search','export')),
    target_ref         TEXT NOT NULL,
    source_name        TEXT NOT NULL,
    schedule_mode      TEXT NOT NULL CHECK (schedule_mode IN
('manual','recurring','webhook','backfill')),
    status             TEXT NOT NULL CHECK (status IN
('queued','running','succeeded','failed','deadletter')),
    priority           SMALLINT NOT NULL DEFAULT 50,
    started_at         TIMESTAMPTZ,
    finished_at        TIMESTAMPTZ,
    error_code         TEXT,
    error_message      TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_collection_jobs_status_priority
ON collection_jobs (status, priority DESC, created_at ASC);

```

For high-volume analytics, mirror event streams into ClickHouse:

```

CREATE TABLE analytics_events
(
    tenant_id UInt64,
    subject_account_id UInt64,
    media_item_id UInt64,
    event_date Date DEFAULT toDate(event_time),
    event_time DateTime64(3, 'UTC'),
    event_type LowCardinality(String),
    related_account_username String,
    confidence_score Float32,
    source_name LowCardinality(String),
    payload_json String
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(event_date)
ORDER BY (tenant_id, subject_account_id, event_type, event_time);

CREATE MATERIALIZED VIEW mv_daily_account_metrics
ENGINE = SummingMergeTree
PARTITION BY toYYYYMM(metric_date)
ORDER BY (tenant_id, subject_account_id, metric_date)
AS
SELECT
    tenant_id,
    subject_account_id,
    toDate(event_time) AS metric_date,
    countIf(event_type = 'comment_inbound') AS comments_inbound,
    countIf(event_type = 'mention_inbound') AS mentions_inbound,
    countIf(event_type = 'like_outbound') AS likes_outbound,
    countIf(event_type = 'follow_edge_added') AS follows_added,
    countIf(event_type = 'follow_edge_removed') AS follows_removed
FROM analytics_events
GROUP BY tenant_id, subject_account_id, metric_date;

```

Recommended API design

The external API should be boring, explicit, and multi-tenant. Use OAuth/OIDC for user auth, workspace-scoped JWTs for the frontend, and service-to-service mTLS or signed tokens internally.

Endpoint	Method	Purpose
<code>/v1/tenants/{tenantId}/accounts</code>	POST	Register a connected or monitored account
<code>/v1/tenants/{tenantId}/accounts/search</code>	GET	Search public professional accounts / tracked entities

Endpoint	Method	Purpose
<code>/v1/accounts/{accountId}</code>	GET	Canonical account profile and latest metrics
<code>/v1/accounts/{accountId}/timeline</code>	GET	Unified activity stream with filters
<code>/v1/accounts/{accountId}/media</code>	GET	Recent media and rollups
<code>/v1/accounts/{accountId}/interactions/inbound</code>	GET	Mentions, comments, messages, audience interactions
<code>/v1/accounts/{accountId}/interactions/outbound</code>	GET	Outbound actions attributable to the account
<code>/v1/accounts/{accountId}/snapshots</code>	GET	Historical follower/following/media snapshots
<code>/v1/ads/library/search</code>	POST	Public ad archive search where available
<code>/v1/jobs</code>	POST	Queue refresh, backfill, export, or discovery jobs
<code>/v1/jobs/{jobId}</code>	GET	Poll job state
<code>/v1/alerts/rules</code>	POST	Save watch rules and anomaly alerts
<code>/v1/webhooks/meta/instagram</code>	POST	Meta webhook ingress
<code>/v1/exports/accounts/{accountId}</code>	POST	Generate CSV/JSON/Parquet exports

Example payloads:

```
POST /v1/tenants/42/accounts
{
  "username": "examplebrand",
  "mode": "connected",
  "connection": {
    "provider": "meta_instagram_login",
    "access_token": "short_lived_token_here"
  }
}
```

```
GET /v1/accounts/981/timeline?from=2026-06-01T00:00:00Z&to=2026-06-22T23:59:59Z&types=comment_inbound,mention_inbound,follow_edge_added
{
  "account_id": 981,
  "username": "examplebrand",
  "events": [
```

```

{
  "event_time": "2026-06-22T08:14:11Z",
  "event_type": "mention_inbound",
  "related_account_username": "creator_xyz",
  "confidence_score": 1.0,
  "source_name": "meta_webhook",
  "payload": {
    "media_id": "1789...",
    "caption_excerpt": "Loved this drop @examplebrand"
  }
},
"next_cursor": "eyJvZmZzZXQiOjEwMDB9"
}

```

```

POST /v1/ads/library/search
{
  "search_term": "examplebrand",
  "countries": ["GB", "DE", "FR"],
  "platforms": ["instagram"],
  "date_from": "2026-01-01",
  "date_to": "2026-06-22"
}

```

Dashboard wireframe and metrics

A high-performing agency dashboard should emphasize **freshness, comparability, and actionability** instead of raw data exhaust.

Account: @examplebrand	Freshness: 2m ago	Workspace: ACME
KPI Strip: Followers Following Media Avg ER 7d Growth 30d Growth		
Growth & Engagement Trend	Unified Activity Feed	
follower delta, ER, mentions	follows, unfollows, comments, mentions, ads	
Recent Media Grid	Inbound vs Outbound Interaction Split	
reels/posts/stories/cards	comments in, mentions, likes/comments out	
Top Commenters / Top Mentions	Competitor Benchmark Table	

verified, repeat, sentiment	peers by growth, ER, posting frequency
Ad / Sponsored Content Panel	Alerts / Anomalies
archive hits, paid boosts	growth spike, follow spike, engagement drop

Recommended aggregate metrics:

Category	Metrics
Audience	current followers, following, follower delta 1d/7d/30d, velocity, acceleration
Content	media count, posting frequency, media mix, top post by likes/comments/views
Engagement	average likes, comments, views, ER by post and by view, comment velocity
Inbound	unique commenters, mention count, repeat commenters, verified engagement share
Outbound	likes/comments/replies performed by owned account; inferred outbound public activity where policy allows
Benchmarking	percentile rank vs peer set, competitor share-of-engagement, posting cadence delta
Paid / promo	ad archive hits, sponsored-post prevalence, paid-vs-organic comparison where available
Operations	source freshness, source confidence, webhook lag, job failure rate, quota burn rate

Migration, ETL, and retention strategy

Use a layered ETL model:

Layer	Purpose	Retention
Raw payloads	Original API/provider/scrape responses in object storage	7-30 days by default
Normalized staging	Parsed records with schema version and provenance	30-90 days
Canonical relational entities	Active entities, jobs, rules, memberships, current state	As long as needed for service
Analytical event store	Time-series activity events and snapshots	90-365 days identifiable; longer if de-identified
Aggregates / rollups	Daily/weekly/monthly metrics, anomaly baselines	Multi-year

Layer	Purpose	Retention
Compliance tombstones	Suppression lists, deletion audit metadata	Per legal/compliance requirement

The operational migration pattern should be **database migrations first, event-schema versioning second, dual-write or CDC third**. PostgreSQL logical replication plus Debezium makes it realistic to move mutations into Kafka-compatible topics and then into ClickHouse and search indexes without in-app fan-out complexity. PostgreSQL partitioning plus `pg_partman` simplifies lifecycle management for large event tables. ⁴⁵

Delivery roadmap, cost model, and scaling path

The implementation roadmap should prioritize a product that can actually survive platform and legal review.

Prioritized roadmap and milestones

Phase	Goal	Deliverables
Foundation	Establish compliant control plane	OIDC auth, tenancy/RBAC, account model, ingestion scheduler, PostgreSQL schema, audit logging
Official MVP	Launch agency-safe value	Connected pro-account auth, media + insights sync, comments/mentions webhooks, dashboard KPIs, exports
Competitive intelligence	Add public professional-account research	Business Discovery ingestion, hashtag workflows, peer benchmarking, watchlists, alerting
Ad intelligence	Add paid-social workflows	Owned ads integration, public Ad Library search panel, timeline overlay for paid/organic
Analytics scale-up	Improve latency and depth	ClickHouse deployment, materialized rollups, anomaly detection, historical backfills
Provider enrichment	Fill gaps selectively	Vendor adapter SDK, provenance labels, confidence scoring, legal hold/takedown workflows
Public-intelligence ring	Optional and high-risk	Separate pipeline, access controls, jurisdiction gating, abuse detection, stricter retention

A sensible milestone sequence is:

- **Month 1–2:** platform foundation, auth, tenancy, PostgreSQL schema, Meta webhook ingress, CI/CD.
- **Month 3–4:** connected-account dashboards, comments/mentions, media sync, exports, alerting.
- **Month 5–6:** Business Discovery competitor views, hashtag discovery, benchmark reports, ClickHouse rollups.
- **Month 7–9:** ads module, provider abstraction, anomaly models, agency-facing workflow polish.

- **Only after this:** any Dolphin-Radar-style public-activity reconstruction, and only behind a separate risk program.

Indicative monthly cost model

The table below is an **illustrative AWS-style model**, not a quote. It uses public list prices such as EKS control plane pricing, EC2 c7i instance pricing, RDS r7g instance pricing, S3 standard storage pricing, and NAT gateway pricing. ⁴⁶

Stage	Typical workload	Infra shape	Indicative infra cost	Data acquisition cost
Prototype	500–2,000 tracked accounts, mostly official APIs	1 EKS cluster, 3× c7i.xlarge app nodes, 1× db.r7g.large, Redis, 2 TB S3, 1 NAT	~\$900–\$2,000/mo	\$0–\$1,500/mo
Agency launch	10k–25k tracked accounts, mixed official + enrichment	EKS, 4–6 app nodes, 2–3 ClickHouse nodes, db.r7g.xlarge, Redis, 10 TB S3	~\$3,000–\$8,000/mo	\$2,500–\$15,000+/mo
Large-scale public intelligence	100k+ watched accounts, frequent refreshes	Larger Kubernetes cluster, larger ClickHouse footprint, bigger RDS, more object storage, higher egress	~\$15,000–\$50,000+/mo	Often dominant; can exceed infra by 2–5×

A few anchor numbers are worth calling out. Amazon EKS standard support is **\$0.10 per cluster-hour**; S3 Standard is **\$0.023/GB-month** for the first 50 TB; a c7i.xlarge in us-east-1 is about **\$0.178/hour** or roughly **\$130/month**; a c7i.2xlarge is about **\$0.357/hour**; db.r7g.large is about **\$0.239/hour** and db.r7g.xlarge about **\$0.478/hour**; NAT Gateway pricing includes **\$0.045/hour** plus **\$0.045/GB** processed in the AWS examples. ⁴⁷

For third-party data, Bright Data publicly lists Instagram datasets at **\$250 per 100,000 records**, while Apify's maintained Instagram scrapers are listed from about **\$1.00 per 1,000 posts** and **\$1.50 per 1,000 results** on several actors. That means a million public records can quickly cost **\$1,000–\$2,500+** before any of your own compute, QA, legal, or customer-support overhead. ⁵

Scaling plan

At small scale, a single regional deployment is enough. At medium scale, separate **ingestion**, **query**, and **export** workloads on Kubernetes. At larger scale, introduce:

- one topic family per source and data class,
- per-tenant and per-source quotas in Redis,
- adaptive refresh intervals based on account importance and freshness SLA,
- hot/warm/cold storage tiers,
- ClickHouse pre-aggregates for KPI cards,

- delayed or budget-capped enrichment jobs,
- and a replayable dead-letter/recovery process.

The most important optimization is not CPU; it is **selective collection**. Tracking every public account at a uniform cadence is economically irrational. Instead, score accounts by customer value, campaign windows, anomaly likelihood, and recent change rate, then allocate crawl budget accordingly. That is the difference between a viable agency platform and a scraper bill with a logo on it. ⁴⁸

Bottom-line recommendation

If the goal is a serious, scalable, enterprise-sellable product, build this as an **Instagram intelligence platform for agencies and connected brands**, not as a generalized hidden-activity surveillance product for arbitrary individuals. The official Meta ecosystem supports a substantial and commercially attractive subset of the opportunity: professional-account analytics, public professional-account discovery, hashtags, comments, mentions, messaging, creator workflows, and limited ad intelligence. The farther you move toward “who did this arbitrary public user follow or like most recently,” the more your architecture, cost structure, and legal posture become dominated by scraping risk rather than software quality. ⁴⁹

The strongest architecture, therefore, is a **dual-track platform** with a compliant core and a separately governed experimental public-intelligence layer. The strongest database strategy is **PostgreSQL + ClickHouse**, with DuckDB for analyst-side work and Redis plus Kafka-compatible streaming around them. The strongest product roadmap is to ship **official-account value first**, then add public professional-account research, then ads, then only carefully gated enrichment. That path is both the most defensible and the most likely to compound into a durable business. ⁵⁰

-
- ¹ ²² [Update on Instagram Basic Display API - Meta for Developers](https://developers.facebook.com/blog/post/2024/09/04/update-on-instagram-basic-display-api/?utm_source=chatgpt.com)
https://developers.facebook.com/blog/post/2024/09/04/update-on-instagram-basic-display-api/?utm_source=chatgpt.com
 - ² ⁷ <https://www.dolphinradar.com/instagram-tracker>
<https://www.dolphinradar.com/instagram-tracker>
 - ³ ¹⁴ ²⁶ ⁴⁹ <https://developers.facebook.com/docs/instagram-platform/instagram-api-with-instagram-login/>
<https://developers.facebook.com/docs/instagram-platform/instagram-api-with-instagram-login/>
 - ⁴ ⁴⁰ ⁴⁵ <https://www.postgresql.org/docs/current/logical-replication.html>
<https://www.postgresql.org/docs/current/logical-replication.html>
 - ⁵ ³¹ <https://brightdata.com/products/datasets/instagram>
<https://brightdata.com/products/datasets/instagram>
 - ⁶ <https://www.dolphinradar.com/>
<https://www.dolphinradar.com/>
 - ⁸ <https://snoopreport.com/>
<https://snoopreport.com/>
 - ⁹ [Instagram Profile Analyzer – IG Account Activity Analytics](https://inflat.com/tools/profile-analyzer/?utm_source=chatgpt.com)
https://inflat.com/tools/profile-analyzer/?utm_source=chatgpt.com

¹⁰ <https://www.modash.io/>

<https://www.modash.io/>

¹¹ <https://hypeauditor.com/reports/instagram/>

<https://hypeauditor.com/reports/instagram/>

¹² <https://socialblade.com/instagram>

<https://socialblade.com/instagram>

¹³ <https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/creator-marketplace/>

<https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/creator-marketplace/>

¹⁵ ²⁰ ³³ https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-user/business_discovery/

https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-user/business_discovery/

¹⁶ ²³ ²⁸ **Comments - Instagram Platform - Meta for Developers**

[https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-media/comments/?](https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-media/comments/?utm_source=chatgpt.com)

[utm_source=chatgpt.com](https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-media/comments/?utm_source=chatgpt.com)

¹⁷ <https://developers.facebook.com/docs/graph-api/webhooks/reference/instagram/>

<https://developers.facebook.com/docs/graph-api/webhooks/reference/instagram/>

¹⁸ <https://developers.facebook.com/docs/instagram-platform/instagram-api-with-instagram-login/messaging-api/>

<https://developers.facebook.com/docs/instagram-platform/instagram-api-with-instagram-login/messaging-api/>

¹⁹ <https://developers.facebook.com/docs/permissions/>

<https://developers.facebook.com/docs/permissions/>

²¹ <https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-user/user-likes/>

<https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-user/user-likes/>

²⁴ **Overview - Instagram Platform - Meta for Developers - Facebook**

https://developers.facebook.com/docs/instagram-platform/overview/?utm_source=chatgpt.com

²⁵ ²⁹ **Meta Ad Library API**

https://www.facebook.com/ads/library/api/?utm_source=chatgpt.com

²⁷ https://developers.facebook.com/docs/features-reference/instagram-public-content-access?locale=fr_FR

https://developers.facebook.com/docs/features-reference/instagram-public-content-access?locale=fr_FR

³⁰ <https://docs.getphylo.com/docs/api-reference/FAQs/platforms/instagram>

<https://docs.getphylo.com/docs/api-reference/FAQs/platforms/instagram>

³² ³⁴ **Terms of Use**

https://help.instagram.com/581066165581870/?utm_source=chatgpt.com

³⁵ **Regulation - 2016/679 - EN - gdpr - EUR-Lex - European Union**

https://eur-lex.europa.eu/eli/reg/2016/679/oj?utm_source=chatgpt.com

³⁶ <https://www.reuters.com/legal/litigation/anti-hacking-law-does-not-bar-data-scraping-public-websites-9th-circuit-2022-04-19/>

<https://www.reuters.com/legal/litigation/anti-hacking-law-does-not-bar-data-scraping-public-websites-9th-circuit-2022-04-19/>

37 <https://blog.snoopreport.com/everything-you-need-to-know-about-instagram-activity-history/>
<https://blog.snoopreport.com/everything-you-need-to-know-about-instagram-activity-history/>

38 Why your account has been restricted for data scraping ...
https://help.instagram.com/740480200552298/?utm_source=chatgpt.com

39 50 <https://clickhouse.com/docs/intro>
<https://clickhouse.com/docs/intro>

41 <https://kafka.apache.org/documentation/>
<https://kafka.apache.org/documentation/>

42 <https://docs.rs/axum/latest/axum/>
<https://docs.rs/axum/latest/axum/>

43 <https://opentelemetry.io/docs/>
<https://opentelemetry.io/docs/>

44 <https://www.postgresql.org/docs/current/brin.html>
<https://www.postgresql.org/docs/current/brin.html>

46 47 <https://aws.amazon.com/eks/pricing/>
<https://aws.amazon.com/eks/pricing/>

48 <https://redis.io/docs/latest/develop/use-cases/rate-limiter/>
<https://redis.io/docs/latest/develop/use-cases/rate-limiter/>